Introduction to TypeScript and Node.js

A practical introductory guide to Node.js, npm, Yarn, pnpm, TypeScript project setup, dependencies, compilation, and execution.

1. The Tools and How They Fit Together

The most important thing for a beginner is to separate the role of each tool. They are related, but they are not the same thing.

```
JavaScript / TypeScript source
        │
        ├── TypeScript compiler (tsc) → JavaScript
        │
        └── Node.js → runs JavaScript outside the browser
                │
                └── package managers: npm / Yarn / pnpm
```

Tool	Purpose
Node.js	Runs JavaScript outside a web browser.
npm	Package manager bundled with Node.js.
Yarn	Alternative JavaScript package manager.
pnpm	Alternative package manager with efficient shared package storage.
TypeScript	JavaScript plus static type checking; normally compiled to JavaScript.
tsc	The TypeScript compiler command.
package.json	Project metadata, scripts, dependencies, and development dependencies.

2. Install Node.js

Install the current Node.js LTS release from the official Node.js website. npm is installed together with Node.js.

Verify the installation

```
node --version
npm --version
```

If both commands print version numbers, Node.js and npm are available in your terminal.

Tip: after installing Node.js on Windows, close and reopen PowerShell or VS Code so the updated PATH is loaded.

3. Windows PowerShell: Script Execution Policy

On Windows, PowerShell may block command shims such as npm.ps1, npx.ps1, yarn.ps1, or pnpm.ps1. A typical error says that running scripts is disabled on the system.

Temporary bypass for the current terminal only

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

This is a convenient classroom/lab option because it affects only the current PowerShell process. Closing that terminal removes the temporary setting.

User-level option

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Check current settings with:

```
Get-ExecutionPolicy -List
```

Note: Avoid changing LocalMachine or using Unrestricted unless you specifically understand and need the security implications. Process-scope Bypass is usually enough for a temporary lab session.

4. Create a Project

Create a folder, enter it, and initialize a Node.js project. Choose one package manager for the project.

```
mkdir typescript-demo
cd typescript-demo
```

Using npm

```
npm init -y
```

Using Yarn

```
yarn init
```

Using pnpm

```
pnpm init
```

The result is a package.json file. It describes the project and records installed packages and scripts.

```
{
  "name": "typescript-demo",
  "version": "1.0.0",
  "scripts": {},
  "dependencies": {},
  "devDependencies": {}
}
```

5. Dependencies vs Dev Dependencies

Dependency

A dependency is a package the application needs as part of its normal runtime behavior. For example, a server that imports Express needs Express as an application dependency.

```
npm install express
yarn add express
pnpm add express
```

Dev dependency

A dev dependency is primarily required while developing, compiling, linting, testing, or formatting the project. TypeScript is commonly a dev dependency because the compiler is used to build the source code.

```
npm install -D typescript
yarn add -D typescript
pnpm add -D typescript
```

Category	Typical examples	Why

dependencies	express, axios, react	Used by the application itself.
devDependencies	typescript, eslint, prettier, vitest	Used for development/build/test tooling.

Note: Rule of thumb: if the package is only needed to create, test, or build the program, it usually belongs in devDependencies.

6. npm vs Yarn vs pnpm Command Cheat Sheet

Task	npm	Yarn	pnpm
Initialize	npm init -y	yarn init	pnpm init
Install all	npm install	yarn install	pnpm install
Add package	npm install axios	yarn add axios	pnpm add axios
Add dev package	npm install -D typescript	yarn add -D typescript	pnpm add -D typescript
Remove package	npm uninstall axios	yarn remove axios	pnpm remove axios
Update	npm update	yarn upgrade	pnpm update
Run script	npm run dev	yarn dev	pnpm dev
Run local binary	npx tsc	yarn tsc	pnpm exec tsc
List packages	npm list	yarn list	pnpm list

Do not casually mix package managers

Each package manager normally creates its own lock file. Use one package manager consistently in a project.

Package manager	Lock file
npm	package-lock.json
Yarn	yarn.lock
pnpm	pnpm-lock.yaml

7. Install Yarn or pnpm (Optional)

npm already comes with Node.js. If you want another package manager, modern Node installations can use Corepack when available.

```
corepack enable
```

Check Yarn

```
yarn --version
```

Check pnpm

```
pnpm --version
```

A common alternative for pnpm is:

```
npm install -g pnpm
pnpm --version
```

When joining an existing project, use the package manager and version expected by that project rather than changing it arbitrarily.

8. Install TypeScript for a Project

Prefer installing TypeScript locally in the project. This keeps the compiler version tied to the project instead of depending on whatever happens to be installed globally.

npm

```
npm install -D typescript
npx tsc --version
```

Yarn

```
yarn add -D typescript
yarn tsc --version
```

pnpm

```
pnpm add -D typescript
pnpm exec tsc --version
```

A global install such as npm install -g typescript can make tsc available system-wide, but local project installation is generally more reproducible.

9. Write, Compile, and Run Your First TypeScript Program

Step 1 — create index.ts

```
const name: string = "Fuad";
const age: number = 22;

console.log(`Hello ${name}`);
console.log(`Age: ${age}`);
```

Step 2 — compile TypeScript into JavaScript

```
npx tsc index.ts
```

This produces index.js next to index.ts.

```
index.ts  --tsc-->  index.js
```

Step 3 — run the JavaScript with Node.js

```
node index.js
```

Core mental model:

```
TypeScript (.ts)
      │
      │ tsc
      ▼
JavaScript (.js)
      │
      │ node
      ▼
Program runs
```

10. Configure TypeScript with tsconfig.json

For a real project, create a TypeScript configuration instead of compiling individual files manually.

```
npx tsc --init
```

Equivalent with pnpm:

```
pnpm exec tsc --init
```

Simple beginner configuration

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "CommonJS",
    "strict": true,
    "rootDir": "./src",
    "outDir": "./dist"
  },
  "include": ["src"]
}
```

Recommended project structure:

```
typescript-demo/
├── src/
│   └── index.ts
├── dist/
│   └── index.js
├── package.json
├── package-lock.json   (or yarn.lock / pnpm-lock.yaml)
└── tsconfig.json
```

Now compile the whole configured project with:

```
npx tsc
```

Then run:

```
node dist/index.js
```

11. package.json Scripts, Watch Mode, and tsx

Build scripts

```
{
  "scripts": {
    "build": "tsc",
    "start": "node dist/index.js"
  }
}
```

Run them with:

```
npm run build
npm start
```

With pnpm or Yarn:

```
pnpm build
pnpm start
```

```
yarn build
yarn start
```

TypeScript watch mode

```
npx tsc --watch
```

The compiler keeps running and recompiles when files change. Stop it with Ctrl+C.

Run TypeScript directly during development with tsx

```
npm install -D tsx
# or: pnpm add -D tsx
# or: yarn add -D tsx
```

```
npx tsx src/index.ts
npx tsx watch src/index.ts
```

A convenient development script:

```
{
  "scripts": {
    "dev": "tsx watch src/index.ts",
    "build": "tsc",
    "start": "node dist/index.js"
  }
}
```

12. TypeScript Language Basics

Variables and primitive types

```
const username: string = "Fuad";
let age: number = 22;
const active: boolean = true;
```

TypeScript often infers the type, so this is also valid:

```
let name = "Fuad";   // inferred as string
let score = 90;       // inferred as number
```

Arrays

```
const numbers: number[] = [1, 2, 3];
const names: string[] = ["Fuad", "Alice"];
```

Functions

```
function add(a: number, b: number): number {
  return a + b;
}

console.log(add(10, 20));
```

void

```
function greet(name: string): void {
  console.log(`Hello ${name}`);
}
```

Objects and type aliases

```
type Student = {
  id: number;
  name: string;
  cgpa: number;
};

const student: Student = {
  id: 1,
  name: "Fuad",
  cgpa: 3.8
};
```

Interfaces

```
interface User {
  name: string;
  age?: number;   // optional
}

const user: User = { name: "Alice" };
```

Union types

```
let id: string | number;

id = "A101";
id = 101;
```

Literal types

```
type Status = "loading" | "success" | "error";

let status: Status = "loading";
```

unknown vs any

```
let value: unknown = "hello";

if (typeof value === "string") {
  console.log(value.toUpperCase());
}
```

Use any sparingly because it disables much of TypeScript's type checking. unknown is safer when you truly do not know a value's type.

13. Common Errors and Fixes

Problem	Likely fix
node is not recognized	Install Node.js, then reopen the terminal/VS Code.
npm.ps1 cannot be loaded	Use Process-scope PowerShell bypass or an appropriate CurrentUser policy.
tsc is not recognized	Install TypeScript locally, then use npx tsc / pnpm exec tsc / yarn tsc.

Cannot find module	Run your package manager install command and check the import/package name.
JavaScript file is not in dist	Check rootDir/outDir/include in tsconfig.json, then run tsc.
Multiple lock files	Pick the intended package manager and remove unintended lock files only after confirming project policy.
node_modules is huge	Normal for npm/Yarn projects; pnpm reduces duplicated package storage.
Type error during compilation	Read the reported file/line and make the value match the declared/inferred type.

14. One-Page Command Reference

Install/check

```
node -v
npm -v
yarn -v
pnpm -v
```

Create a project

```
mkdir ts-demo
cd ts-demo
npm init -y
```

Install TypeScript

```
npm install -D typescript
# Yarn: yarn add -D typescript
# pnpm: pnpm add -D typescript
```

Initialize TypeScript config

```
npx tsc --init
```

Compile and run

```
npx tsc
node dist/index.js
```

Development runner

```
npm install -D tsx
npx tsx src/index.ts
npx tsx watch src/index.ts
```

Useful package commands

```
npm install
npm install <package>
npm install -D <package>
npm uninstall <package>
npm update
npm run <script>
```

PowerShell temporary bypass

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Note: Remember the core pipeline: write TypeScript in src/ → compile with tsc → JavaScript appears in dist/ → run the JavaScript with node. npm, Yarn, and pnpm mainly differ in how packages are installed and managed; choose one per project.